

Lab 04 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — `COPY ../gemeenschappelijk/...`

Antwoord. De build heeft alleen toegang tot de **context** — de map die als argument meegegeven is (`.`, dus `~/projecten/api/`). `../gemeenschappelijk` ligt erbuiten: Buildah brengt het pad terug binnen de context (`possible escaping context directory`), vindt er niets, en faalt. `-f` wijst alleen de Dockerfile aan, niet de perimeter; een absoluut pad wordt eveneens binnen de context teruggebracht; `sudo` verandert niets aan een perimeterprobleem, dat geen rechtenprobleem is. Oplossing: bouwen vanuit de bovenliggende map (`podman build -f api/Dockerfile -t api:1.0 ~/projecten`) met paden `COPY api/... gemeenschappelijk/...`, of `config.yml` vóór de build naar het project kopiëren — of beter, het helemaal niet meenemen en het bij de uitvoering injecteren (lab 08).

Waarom. De context is een grens van veiligheid en reproduceerbaarheid: een Dockerfile kan alleen afhangen van wat men hem expliciet geeft. Bij Docker is dat fysiek (de context wordt gearchiveerd en naar de daemon gestuurd); bij Podman is het een regel die Buildah afdwingt — zelfde resultaat.

Nuance. Podman aanvaardt meerdere benoemde contexten: `podman build --build-context gemeenschappelijk=../gemeenschappelijk .` en dan `COPY --from=gemeenschappelijk config.yml /app/`. Dat is de nette oplossing wanneer een gedeeld bestand echt in meerdere images moet.

Voorbeeld.

```
podman build -f Dockerfile.buiten-context -t poging .
# Error: building at STEP "COPY ../gemeenschappelijk/secret.txt /": ... possible escaping context directory error
```

Vraag 2 — `transferring context`, en dan niets meer

Antwoord. Onder Docker verpakt de client de **hele** map (1,1 GB) en stuurt hij ze naar de daemon vóór de eerste instructie: dat is de `transferring context`. Onder Podman leest Buildah de map ter plaatse, zonder archief of overdracht: de traagheid verdwijnt. Maar het tweede risico is intact: een `COPY . .` stopt `node_modules` en `.git` **in de image** — 1,1 GB nutteloze inhoud, plus de volledige Git-geschiedenis (met haar eventuele geheimen) aangeboden aan wie de image downloadt. `.dockerignore` blijft dus verplicht; het dient niet meer de snelheid, maar de inhoud.

Waarom. De context heeft twee rollen: wat *verzonden* wordt (alleen Docker) en wat *kopieerbaar* is. Podman schraapt de eerste kost, niet de tweede.

Nuance. `.git` in een image is een frequent en ernstig lek: de geschiedenis bevat vaak credentials die "sindsdien" verwijderd zijn. En zonder `.dockerignore` maakt een gewijzigd bestand in `node_modules` ook de cache van de `COPY` ongeldig.

Voorbeeld.

```
podman build -q -f Dockerfile.alles -t t .          # image van 218 MB met node_modules
printf 'node_modules\n.git\n' > .dockerignore
podman build -q -f Dockerfile.alles -t t2 .        # 8,7 MB
```

Vraag 3 — EXPOSE en de poort die niet antwoordt

Antwoord. Nee. EXPOSE is een **verklaring**: het documenteert dat de applicatie op 8080 luistert en voedt podman ps en -P. Het maakt geen enkele doorsturing vanaf de host aan. Zonder -p 8080:8080 is de poort alleen bereikbaar vanuit het containernetwerk.

Waarom. Een poort publiceren is een uitrolbeslissing (welke hostpoort, welke interface), geen eigenschap van de image. De image zegt "ik luister op 8080"; de beheerder beslist "ik stel ze bloot op 18080".

Nuance. -P (hoofdletter) publiceert automatisch alle EXPOSE-poorten op willekeurige hostpoorten: daar wordt de verklaring nuttig. En in rootless-modus faalt -p 80:8080 (geprivilegieerde poort); kies ≥ 1024 of stel net.ipv4.ip_unprivileged_port_start in.

Voorbeeld.

```
podman run -d --name a mijn-api:1.0 && curl -m 2 localhost:8080      # mislukt
podman run -d --name b -p 8080:8080 mijn-api:1.0 && curl localhost:8080/actuator/health #
{"status":"UP"}
podman port b                                                         # 8080/tcp -> 0.0.0.0:8080
```

Vraag 4 — RUN java tegenover CMD java

Antwoord. A start de API **tijdens de build**: RUN voert het commando uit op het moment van de constructie, de API start, eindigt nooit... en de build blijft hangen (of, als de API stopt, bevat de image alleen een nutteloze laag). B is correct: CMD legt het commando vast dat bij podman run gestart wordt.

Waarom. RUN dient om het bestandssysteem voor te bereiden (installeren, compileren, kopiëren); CMD / ENTRYPOINT beschrijven het hoofdproces van de toekomstige container. De twee verwarren is constructie en uitvoering verwarren.

Nuance. B zou nog beter zijn met ENTRYPOINT + CMD (vraag 5) en een USER. En een RUN java -jar heeft een legitiem gebruik: een **eindige taak** starten bij de build, zoals java -Djarmode=tools -jar app.jar extract (lab 05).

Voorbeeld.

```
podman build -f A -t a .          # STEP 3/3: RUN java -jar ... - komt nooit tot een einde
podman build -f B -t b . && podman run -d -p 18080:8080 b
```

Vraag 5 — ENTRYPOINT + CMD tegenover CMD alleen

Antwoord. A: podman run img → java -jar /app/api.jar --spring.profiles.active=prod; podman run img --debug → java -jar /app/api.jar --debug (de CMD wordt vervangen, de ENTRYPOINT blijft). B: podman run img → hetzelfde volledige commando; podman run img --debug → voert --debug op zichzelf uit, zonder java: fout executable file not found. Alleen B laat nog podman

`run img sh` toe (de hele `CMD` wordt vervangen door `sh`). Met A start `podman run img sh java -jar api.jar sh`; je hebt `podman run --entrypoint sh img` nodig.

Waarom. De argumenten van `run` vervangen de `CMD` en worden toegevoegd aan de `ENTRYPOINT`. A is gemaakt voor een "applicatie"-image, B voor een "tool"-image.

Nuance. In exec-vorm zetten zowel A als B `java` als PID 1. Een gangbare bedrijfsvariant: `ENTRYPOINT ["java","-jar","app.jar"]` zonder `CMD`, en configuratie via omgevingsvariabelen — argumenten dienen alleen om te debuggen.

Voorbeeld.

```
timeout 5 podman run --rm api-lab:1.0 --debug | head -1      # Arguments recus : --debug
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo ok'    # ok
```

Vraag 6 — Tien seconden, `resorting to SIGKILL`, geen hooks

Antwoord. `CMD java -jar /app/api.jar` is een **shell**-vorm: de engine voert `/bin/sh -c "java -jar /app/api.jar"` uit. Op een Debian/Ubuntu-basis is `/bin/sh` `dash`, dat Java als kind start en PID 1 blijft. `podman stop` stuurt `SIGTERM` naar PID 1 — de shell — die het niet doorgeeft. Java ontvangt niets, zijn *shutdown hooks* worden niet uitgevoerd; na 10 seconden kondigt Podman `resorting to SIGKILL` aan en doodt alles (137). Correctie: `CMD ["java","-jar","/app/api.jar"]`. Op Alpine is `/bin/sh` `ash` (busybox), dat **zichzelf vervangt** door het commando wanneer dat eenvoudig is: Java wordt PID 1, ontvangt `SIGTERM`, en het probleem is onzichtbaar in de test.

Waarom. Een POSIX-shell heeft geen enkele verplichting om signalen door te geven aan zijn kinderen; `dash` doet het niet. De exec-vorm schrapt de shell, en dus de vraag.

Nuance. Zelfs Alpine redt een shell-`CMD` met `&&`, `|` of een variabele niet: de shell moet dan blijven. De regel "altijd exec-vorm" vermijdt dat je het gedrag van elke shell moet kennen.

Voorbeeld.

```
podman exec s-deb ps -o pid,args | head -3      # 1 /bin/sh -c java ... 2 java ...
time podman stop s-deb                         # resorting to SIGKILL, 10 s, code 137
```

Vraag 7 — `$JAVA_OPTS` niet geïnterpreteerd

Antwoord. In exec-vorm is er **geen shell**: `$JAVA_OPTS` wordt als zodanig aan Java doorgegeven, als een tekenreeks van zes tekens. Twee correcties: (1) expliciete shell-vorm met `exec` — `ENTRYPOINT ["sh","-c","exec java $JAVA_OPTS -jar /app/api.jar"]`: kost, een afhankelijkheid van `sh` en een minder leesbare regel; (2) de variabele schrappen — Java leest zelf `JAVA_TOOL_OPTIONS` uit de omgeving, dus `ENV JAVA_TOOL_OPTIONS="-Xmx512m"` en `ENTRYPOINT ["java","-jar","/app/api.jar"]`: kost, een boodschap `Picked up JAVA_TOOL_OPTIONS` op `stderr` bij het opstarten, en een variabele die op *alle* Java-processen van de container van toepassing is.

Waarom. Variabelen uitbreiden is een dienst van de shell. De exec-vorm is een reeks argumenten die rechtstreeks aan de systeemaanroep `execve` wordt doorgegeven.

Nuance. Vorm (1) zonder `exec` zou het probleem van vraag 6 opnieuw creëren. En specifiek voor geheugen is `-XX:MaxRAMPercentage=75` beter dan een vaste `-Xmx`: de JVM past zich aan de cgroup aan (lab 10).

Voorbeeld.

```
podman run --rm -e JAVA_TOOL_OPTIONS="-Xmx256m" api-lab:1.0 --debug 2>&1 | head -1
# Picked up JAVA_TOOL_OPTIONS: -Xmx256m
```

Vraag 8 — `--build-arg` en het geheim

Antwoord. Nee. De waarde van een `ARG` zit niet in `Config.Env`, maar ze wordt opgeslagen in de **geschiedenis** van elke instructie die ze gebruikt, en in de buildcache. `podman history --no-trunc api:1.0` toont ze in klare tekst (`|1 DB_PASSWORD=Secr3t! /bin/sh -c ...`). Wie de image heeft, heeft het wachtwoord.

Waarom. De geschiedenis beschrijft hoe elke laag geproduceerd werd, argumenten inbegrepen — dat maakt de cache mogelijk. Een `ARG` is een invoer van de build, dus van de cache, dus van de geschiedenis.

Nuance. De goede praktijk: het geheim heeft niets te zoeken bij de *build*. Als het onmisbaar is (private Maven-repository), maakt `RUN --mount=type=secret,id=settings ...` het beschikbaar tijdens één instructie zonder het ooit in een laag te schrijven (lab 08). En een multi-stage beschermt alleen als het geheim uitsluitend in een weggegooid stage gebruikt wordt (lab 05).

Voorbeeld.

```
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | grep DB_PASSWORD
# |1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build met $DB_PASSWORD" > /trace.txt
```

Vraag 9 — Zes minuten per Maven-build

Antwoord.

```
WORKDIR /app
COPY pom.xml .
RUN mvn -q dependency:go-offline      # download van de afhankelijkheden, gecachet
COPY src ./src
RUN mvn -q package -DskipTests      # alleen compilatie
```

De cache hergebruikt een instructie als haar tekst **en** haar invoer ongewijzigd zijn. `pom.xml` verandert zelden: de laag `dependency:go-offline` (de vijf minuten download) blijft `Using cache`. Alleen de compilatie wordt opnieuw gespeeld wanneer een `.java` verandert. De build blijft traag wanneer `pom.xml` verandert — toevoegen of bijwerken van een afhankelijkheid — aangezien de laag van de afhankelijkheden dan ongeldig gemaakt wordt.

Waarom. `COPY . /app` plaatst *alle* code vóór Maven; elke commit maakt de kopie ongeldig, en dus alles wat volgt.

Nuance. `dependency:go-offline` is niet perfect (sommige plugins downloaden nog bij `package`). Een `RUN --mount=type=cache,target=/root/.m2` (lab 05) bewaart de lokale repository tussen builds, zelfs als `pom.xml` verandert. En bij Podman hangt geen van die winsten af van een daemon: de cache zit in je gebruikersopslag.

Voorbeeld.

```
time podman build -t api-lab:2.1 .      # RUN ... sleep 5: --> Using cache; 0,7 s
```

Vraag 10 — Drie apt-RUN s

Antwoord. (1) **Drie lagen in plaats van één:** de apt-lijsten (`/var/lib/apt/lists` , ~40 MB) worden geschreven in laag 2; laag 3 verbergt ze alleen, de image behoudt de 40 MB. (2)

Geïsoleerde `apt-get update` wordt gecachet: over enkele weken zal een wijziging van de regel `install` verouderde indexen hergebruiken (pakketten niet gevonden, oude versies). (3) **Geen** `--no-install-recommends` en `vim` in een productie-image: tientallen MB nutteloze pakketten, evenveel aanvalsoppervlak. Correcte versie:

```
RUN apt-get update \
&& apt-get install -y --no-install-recommends curl \
&& rm -rf /var/lib/apt/lists/*
```

Waarom. Een laag is onveranderlijk; opruimen heeft alleen effect in de laag die de bestanden aanmaakte. En de cache werkt instructie per instructie: `update` en `install` moeten samen gaan.

Nuance. Op Alpine: `apk add --no-cache curl` doet alles in één regel. En `vim` in een image is nooit gerechtvaardigd: `podman exec` met een tijdelijke editor, of helemaal geen editor (distroless image, lab 05).

Voorbeeld.

```
podman history img --format 'table {{.Size}}\t{{.CreatedBy}}' # de laag "rm -rf" is 0B, die
erboven 40 MB
```

Vraag 11 — Using cache en dan alles herbouwd

Antwoord. De regel: **een ongeldig gemaakte instructie maakt alle volgende ongeldig**, en de cache wordt van boven naar onder gelezen. Dag 1: alleen de laatste `COPY` is veranderd, alles ervoor is identiek → acht `Using cache`, en dan herbouw van de laatste twee. Dag 2: een instructie ingevoegd op de derde positie verandert de tekst van de Dockerfile vanaf regel 3 → stappen 3 tot 10 zijn nieuw, dus herbouwd — zelfs als hun inhoud niet bewogen is.

Waarom. De cachesleutel van een stap is (ouderlaag, instructie, invoer). De ouderlaag veranderen verandert de sleutel van alles wat volgt.

Nuance. Daarom staan variabele `ENV`, `ARG` en `LABEL` (buildnummer, datum) **op het einde** van een Dockerfile, en stabiele metadata vooraan. Een `ARG BUILD_DATE` op regel 2 maakt alles ongeldig, bij elke build.

Voorbeeld.

```
podman build -t api-lab:3.1 . # STEP 3/5: COPY ... (opnieuw) en dan STEP 4/5: RUN ... sleep 5
(ook opnieuw)
```

Vraag 12 — ADD tegenover COPY

Antwoord. Twee gedragingen eigen aan `ADD`: het **pakt** automatisch een lokaal archief uit (`.tar`, `.tar.gz`, `.tar.xz`) naar de bestemming, en het **downloadt** een URL. De officiële aanbeveling is `COPY` omdat die gedragingen impliciet zijn: een `ADD bestand.tar.gz /app/` dat uitpakt terwijl men het archief wilde kopiëren, een URL gedownload zonder verificatie of cache en zonder mogelijke `rm` in dezelfde laag. Het enige gerechtvaardigde geval: een **lokaal** archief uitpakken in één instructie (`ADD rootfs.tar.gz /`).

Waarom. Een Dockerfile moet leesbaar zijn zonder verrassingen; `COPY` doet één ding. Voor een URL is `RUN curl ... && tar ... && rm ...` in één enkele `RUN` expliciet en op te ruimen.

Nuance. Beide aanvaarden `--chown` (en `--chmod`), nuttig vóór een `USER`. En `COPY --from=` (lab 05) heeft geen `ADD`-equivalent.

Voorbeeld.

```
ADD app.tar.gz /opt/          # /opt/app/... uitgepakt
COPY app.tar.gz /opt/         # /opt/app.tar.gz als zodanig
```

Vraag 13 — `USER` te vroeg, en `HUSER 100999`

Antwoord. `USER` geldt voor alle volgende instructies, `RUN` inbegrepen. Geplaatst na `FROM` laat het `apt-get install` en `mkdir` uitvoeren door UID 1000, die geen recht heeft om te schrijven in `/usr`, `/var` of `/`: `Permission denied`. In een goed geschreven Dockerfile staat `USER` **net vóór** `ENTRYPOINT` / `CMD`, na het installeren, het aanmaken van de mappen en het aanpassen van hun eigenaar (`chown`, `COPY --chown`). In rootless-modus wordt UID 1000 van de container op de host geprojecteerd via `/etc/subuid`: de eerste "bijkomende" UID (1) komt overeen met 100000, dus `1000 → 100999`. `USER` blijft nuttig: (a) het ontnemt de applicatie de root-rechten *in* de container (imagebestanden wijzigen, luisteren op 80, pakketten installeren); (b) dezelfde image zal onder Docker of Kubernetes draaien, waar root echt root is; (c) beveiligingsscanners en toelatingsbeleid weigeren images zonder `USER`.

Waarom. De `user`-namespace beschermt de *host*; `USER` beschermt de *container* en wat erin zit. De twee lagen vullen elkaar aan.

Nuance. `USER 1000:1000` zonder de gebruiker aan te maken werkt (Podman voegt zelfs on the fly een `/etc/passwd`-ingang toe), maar sommige programma's willen een `HOME` of een naam: `RUN adduser -D -u 1000 app` en dan `USER app` is robuuster.

Voorbeeld.

```
podman top u user,huser          # 1000 100999
podman run --rm --entrypoint sh api-lab:user-ok -c 'touch /app/x'      # Permission denied: goed
ZO.
```

Vraag 14 — "Eén enkele `RUN`"

Antwoord. Hij heeft gelijk wanneer bestanden aangemaakt door een instructie verwijderd worden door een andere (installatie + opruiming, uitpakken + verwijderen van het archief): gescheiden blijven de bestanden in de image. Hij heeft ongelijk wanneer de groepering stabiel en vluchtig vermengt: één `RUN` die de afhankelijkheden downloadt **en** de code compileert, wordt bij elke commit ongeldig gemaakt en downloadt dus alles opnieuw; en één laag van 300 MB wordt bij elke `push` volledig opnieuw overgedragen, terwijl vijf lagen waarvan vier stabiel alleen het verschil overdragen.

Waarom. Het aantal lagen heeft op zichzelf bijna geen kost. Wat telt, is **welke bestanden in welke laag leven** (grootte) en **hoe vaak elke laag verandert** (cache en overdracht).

Nuance. Praktische regel: één `RUN` per "eenheid van verandering" — systeeminstallatie (stabiel), afhankelijkheden (semi-stabiel), code (vluchtig). Multi-stage (lab 05) en de opdeling van de Spring Boot-JAR passen precies die logica toe.

Voorbeeld.

```
podman history mijn-api:1.0 --format 'table {{.Size}}\t{{.CreatedBy}}' # één laag per eenheid  
van verandering
```